

Parallel Decision Diagram-based Multiobjective Discrete Optimization

Rahul Patel¹, Elias B. Khalil¹

¹Department of Mechanical and Industrial Engineering, University of Toronto, Canada
rm.patel@mail.utoronto.ca, elias.khalil@utoronto.ca

Abstract

Decision diagrams (DDs) have recently emerged as a state-of-the-art technique for solving multiobjective discrete optimization problems by quickly enumerating their Pareto frontier (PF). Existing implementations rely on serial algorithms for PF enumeration, where the primary computational bottleneck arises from repeatedly performing dominance filtering at each node of the DD. This work studies the impact of parallelizing dominance filtering on PF enumeration time. We evaluate the parallel algorithms on benchmark instances of the multiobjective knapsack, set packing, and traveling salesperson problems. Across instance sizes, the maximum speed-up achieved by a GPU variant is $7.49\times$ for MOKP, $15.08\times$ for MOSPP, and $13.60\times$ for MOTSP relative to the state-of-the-art serial implementation. The maximum speed-up achieved by the best parallel CPU variant is $3.36\times$ for MOKP, $2.68\times$ for MOSPP, and $5.75\times$ for MOTSP relative to the same baseline, with the caveat that it is able to solve more instances than the GPU variant. The code is available at <https://anonymous.4open.science/r/PDD>.

1 Introduction

Multiobjective discrete optimization (MODO) (Ehrgott 2005; Ehrgott, Gandibleux, and Przybylski 2016; Miettinen 1999) problems arise naturally in many applications where conflicting criteria need to be considered simultaneously in conjunction with constraints and integer variables. While problem-specific (Vianna and Arroyo 2004) and generic evolutionary heuristics (Deb et al. 2002) have been developed for this problem class, recovering the exact Pareto frontier (PF) is desirable in critical applications such as healthcare, finance, and infrastructure planning. Over the past decade, decision diagrams (DDs) (Ciré 2018) have gained traction as an effective framework for solving discrete problems that can be cast in the language of dynamic programming. As a data structure, the DD is an implicit representation of all feasible solutions to a problem. Naturally, one can enumerate the subset of such feasible solutions that are Pareto-optimal by using the multicriteria-shortest path algorithm on the network represented by a DD (Garroppo, Giordano, and Tavanti 2010). This approach has been shown to be quite efficient in practice, yielding exact Pareto frontiers

rather quickly for problems with 2-7 objectives and tens to hundreds of decision variables (Bergman et al. 2022).

Despite the success of DD-based solvers, extracting the exact Pareto frontier remains computationally demanding. The primary bottleneck lies in the state-space expansion, specifically the localized Pareto dominance filtering performed at each node to discard suboptimal labels. Because dominance checking requires pairwise comparisons among all candidate objective vectors at a node, its worst-case time complexity grows quadratically with the size of the intermediate frontiers. As the number of objectives or the size of the problem increases, this filtering step becomes the bottleneck, limiting the applicability of serial DD solvers.

Simultaneously, the proliferation of graphics processing units (GPUs) due to the rise of deep learning has prompted optimization researchers to reconsider classical algorithms and attempt to parallelize them for improved scalability (Ben-Nun and Hoefler 2019). GPUs are throughput-oriented processors designed to execute thousands of concurrent threads. The massive amount of independent, pairwise vector comparisons required during dominance filtering makes this specific bottleneck highly amenable to the *Single Instruction Multiple Thread* (SIMT) architecture of GPUs.

To address this computational bottleneck, we introduce the first parallel DD-based framework for MODO. Through a careful analysis of the PF enumeration process, we identify opportunities for both node-level and candidate-level parallel computation. We propose static and dynamic resource allocation schemes that take into account the nature of the hardware being used (central processing unit (CPU) or GPU) to efficiently balance the varying workloads across the DD layers.

The main contributions of this paper are summarized as follows:

- We present the first parallel algorithm for exact PF enumeration on DDs, identifying key opportunities for concurrent state expansion and dominance filtering.
- We design and implement novel static and dynamic memory and thread allocation schemes tailored for both multi-core CPUs and massively parallel GPUs, effectively mitigating the hazards of data races and synchronization overhead.
- We conduct extensive computational experiments on

benchmark instances of the multiobjective knapsack, set packing, and traveling salesperson problems with up to seven objectives.

- We demonstrate that our parallelization schemes achieve substantial reductions in wall-clock time compared to the state-of-the-art serial enumeration baseline, achieving speedups of up to $X\times$ on CPUs and $Y\times$ on GPUs.

The remainder of this paper is organized as follows. Section 2 provides the necessary background on MODO, DDs, and GPU architecture. Section 3 details our parallel enumeration methodology and resource allocation schemes. ?? presents the computational evaluation across various problem classes. The related work is provided in ??. Finally, ?? concludes the paper and discusses avenues for future work.

2 Preliminaries

2.1 Multiobjective Discrete Optimization

A MODO problem can be defined as the minimization of an m -dimensional objective vector function $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x}))^\top$ subject to $\mathbf{x} \in \mathcal{X}$. Here, $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$ represents a vector of discrete decision variables, and $\mathcal{X} \subseteq \mathbb{Z}^n$ is the bounded, discrete feasible region. We compare different feasible solutions based on the principle of *Pareto dominance*. For any two feasible solutions $\mathbf{x}^1, \mathbf{x}^2 \in \mathcal{X}$, the solution $\mathbf{x}^1$ is said to Pareto dominate $\mathbf{x}^2$, denoted by $\mathbf{f}(\mathbf{x}^1) \prec \mathbf{f}(\mathbf{x}^2)$, if $f_i(\mathbf{x}^1) \leq f_i(\mathbf{x}^2)$ for all $i \in \{1, \dots, m\}$, and there exists at least one index $j \in \{1, \dots, m\}$ strictly satisfying $f_j(\mathbf{x}^1) < f_j(\mathbf{x}^2)$. An objective vector $\mathbf{y}^* = \mathbf{f}(\mathbf{x}^*)$ associated with a feasible solution $\mathbf{x}^* \in \mathcal{X}$ is defined as a *nondominated point* if there exists no other feasible solution $\mathbf{x} \in \mathcal{X}$ such that $\mathbf{f}(\mathbf{x}) \prec \mathbf{y}^*$. The collection of all such nondominated points across the entire feasible region constitutes the *Pareto frontier* (or nondominated frontier) of the MODO problem. Mathematically, this frontier is expressed as $\mathcal{Y}_N = \{\mathbf{f}(\mathbf{x}) \mid \mathbf{x} \in \mathcal{X}, \nexists \mathbf{x}' \in \mathcal{X} \text{ such that } \mathbf{f}(\mathbf{x}') \prec \mathbf{f}(\mathbf{x})\}$. The fundamental aim in exact multiobjective optimization is to completely enumerate this Pareto frontier, providing decision-makers with the exact trade-offs among the competing criteria.

2.2 Decision Diagrams for Multiobjective Discrete Optimization

A DD is a directed layered acyclic graph $\mathcal{D} = (\mathcal{N}, \mathcal{A})$ defined by a node set $\mathcal{N}$ and an arc set $\mathcal{A}$. The node set is partitioned into $n + 1$ strictly sequential layers, $\mathcal{N} = \mathcal{L}_1 \cup \dots \cup \mathcal{L}_{n+1}$, corresponding to the sequence of the n decision variables. The first layer $\mathcal{L}_1$ contains only the *root node* $\mathbf{r}$, and the final layer $\mathcal{L}_{n+1}$ contains only the *terminal node* $\mathbf{t}$.

The arcs in $\mathcal{A}$ are directed from nodes in layer $\mathcal{L}_j$ to nodes in layer $\mathcal{L}_{j+1}$ for $j \in \{1, \dots, n\}$. Every arc $a \in \mathcal{A}$ originating from layer $\mathcal{L}_j$ possesses two key attributes: an *assignment value* $d(a) \in \mathbb{Z}$, which dictates the value assigned to the decision variable x_j , and an *objective weight* $\mathbf{v}(a) \in \mathbb{R}^m$, which represents the transition's contribution to the m -dimensional objective function. A path $p = (a_1, \dots, a_n)$ from the root $\mathbf{r}$ to the terminal node $\mathbf{t}$ encodes a complete

decision vector $\mathbf{x}(p) = (d(a_1), \dots, d(a_n))^\top \in \mathbb{Z}^n$, with an associated objective vector obtained by accumulating the arc weights along the path $\mathbf{v}(p) = \sum_{i=1}^n \mathbf{v}(a_i)$.

Let $\mathcal{X}_{\mathcal{D}} = \{\mathbf{x}(p) \mid p \text{ is an } \mathbf{r} \text{--} \mathbf{t} \text{ path in } \mathcal{D}\}$ denote the set of solutions encoded by the diagram. A DD is said to be *exact* for a given problem if it represents the feasible region ($\mathcal{X}_{\mathcal{D}} = \mathcal{X}$) and the path evaluations correctly compute the objective function ($\mathbf{v}(p) = \mathbf{f}(\mathbf{x}(p))$ for all paths). In this case, the image of the encoded solutions in the objective space is $\mathcal{Y}_{\mathcal{D}} = \{\mathbf{v}(p) \mid p \text{ is an } \mathbf{r} \text{--} \mathbf{t} \text{ path}\}$. The nondominated vectors extracted from $\mathcal{Y}_{\mathcal{D}}$ constitute the exact Pareto frontier $\mathcal{Y}_N$ of the original MODO problem.

To extract this exact Pareto frontier $\mathcal{Y}_N$ from an exact decision diagram $\mathcal{D}$, we can employ the *coupled enumeration scheme*, which dynamically interleaves top-down and bottom-up layer expansion, as detailed in Algorithm 1. This dynamic layer cutset methodology mitigates intermediate label explosion. Initially, a top-down layer pointer i_{td} is set to layer 1 and a bottom-up pointer i_{bu} is set to layer $n + 1$. At each step, the scheme dynamically decides whether to expand the top-down frontier $\mathcal{Y}^\downarrow$ to layer $i_{td} + 1$ or the bottom-up frontier $\mathcal{Y}^\uparrow$ to layer $i_{bu} - 1$. This process continues until the two frontiers meet at a *cutset* layer c (i.e., $i_{td} = i_{bu}$). Finally, for each node $u \in \mathcal{L}_c$, the algorithm computes the convolution of its top-down and bottom-up labels (Algorithm 1, line 35), taking the pairwise sum of these vectors. The merging of these combined sets and the repeated application of a dominance filter produce the exact Pareto frontier $\mathcal{Y}_N$.

A special case of the coupled enumeration scheme is the *top-down enumeration scheme*, in which the bottom-up layer pointer i_{bu} is fixed to the layer $n + 1$. As such, only operations involving ‘ExpandTopDown’ are executed, and the cutset layer naturally occurs at $c = n + 1$. In this case, upon completion of the final layer processing, the label set $\mathcal{Y}^\downarrow(\mathbf{t})$ at the terminal node directly corresponds to the exact Pareto frontier $\mathcal{Y}_N$. This approach is outlined in Algorithm 2. Note that we did not perform convolution or repeated dominance filtering (Algorithm 1, lines 33 to 37) in this approach.

2.3 Graphics Processing Unit

GPUs are throughput-oriented processors that achieve high performance by executing many lightweight threads in parallel. In contrast, CPUs comprise a small number of complex cores optimized for low-latency execution and sophisticated control flow. This architectural difference makes GPUs particularly effective for data-parallel workloads in which the same operation is applied independently across many inputs (Lindholm et al. 2008). Figure 1 illustrates the GPU execution and memory hierarchy. Computation is launched as a *kernel*, which executes as a large collection of *threads*. Threads are grouped into *thread blocks*, and all blocks in a kernel launch form a *grid* (Nickolls et al. 2008). The GPU schedules thread blocks onto *Streaming Multiprocessors* (SMs), where each SM can host multiple blocks concurrently and executes threads in a *Single Instruction Multiple Thread* (SIMT) manner, where threads follow the same instruction stream while operating on different data.

The memory system is also hierarchical. Threads within an SM have access to fast on-chip storage, including reg-

Algorithm 1 Coupled Pareto Frontier Enumeration

Require: Exact decision diagram $\mathcal{D} = (\mathcal{N}, \mathcal{A})$ with layers $\mathcal{L}_1, \dots, \mathcal{L}_{n+1}$

```

1: procedure FILTERDOMINATED( $\mathcal{Y}$ )  $\triangleright \mathcal{O}(|\mathcal{Y}|^2)$ 
2:   return  $\mathcal{Y} \leftarrow \{\mathbf{y} \in \mathcal{Y} \mid \nexists \mathbf{y}' \in \mathcal{Y} \text{ such that } \mathbf{y}' \prec \mathbf{y}\}$ 
3: end procedure
4: procedure EXPANDTOPDOWN( $j, \mathcal{Y}^\downarrow$ )
5:   for each node  $u \in \mathcal{L}_j$  do
6:     for each arc  $a = (u, v)$  originating from  $u$  do
7:        $\mathcal{Y}^\downarrow(v) \leftarrow \mathcal{Y}^\downarrow(v) \cup (\mathcal{Y}^\downarrow(u) \oplus \{\mathbf{v}(a)\})$ 
8:        $\mathcal{Y}^\downarrow(v) \leftarrow \text{FILTERDOMINATED}(\mathcal{Y}^\downarrow(v))$ 
9:     end for
10:  end for
11:  return  $\mathcal{Y}^\downarrow$ 
12: end procedure
13: procedure EXPANDBOTTOMUP( $j, \mathcal{Y}^\uparrow$ )
14:   for each node  $v \in \mathcal{L}_j$  do
15:     for each arc  $a = (u, v)$  entering  $v$  from  $u \in \mathcal{L}_{j-1}$  do
16:        $\mathcal{Y}^\uparrow(u) \leftarrow \mathcal{Y}^\uparrow(u) \cup (\mathcal{Y}^\uparrow(v) \oplus \{\mathbf{v}(a)\})$ 
17:        $\mathcal{Y}^\uparrow(u) \leftarrow \text{FILTERDOMINATED}(\mathcal{Y}^\uparrow(u))$ 
18:     end for
19:   end for
20:   return  $\mathcal{Y}^\uparrow$ 
21: end procedure
22:  $\mathcal{Y}^\downarrow(u) \leftarrow \emptyset, \mathcal{Y}^\uparrow(u) \leftarrow \emptyset$  for all  $u \in \mathcal{N}$ 
23:  $\mathcal{Y}^\downarrow(\mathbf{r}) \leftarrow \{\mathbf{0}\}, \mathcal{Y}^\uparrow(\mathbf{t}) \leftarrow \{\mathbf{0}\}$   $\triangleright \mathbf{r}$  is root,  $\mathbf{t}$  is terminal
24:  $i_{\text{td}} \leftarrow 1, i_{\text{bu}} \leftarrow n + 1$ 
25: while  $i_{\text{td}} \neq i_{\text{bu}}$  do
26:   Dynamically decide whether to expand top-down or bottom-up
27:   if expanding top-down then
28:     EXPANDTOPDOWN( $i_{\text{td}}, \mathcal{Y}^\downarrow$ ),  $i_{\text{td}} \leftarrow i_{\text{td}} + 1$ 
29:   else
30:     EXPANDBOTTOMUP( $i_{\text{bu}}, \mathcal{Y}^\uparrow$ ),  $i_{\text{bu}} \leftarrow i_{\text{bu}} - 1$ 
31:   end if
32: end while
33:  $c \leftarrow i_{\text{td}}, \mathcal{V}_N \leftarrow \emptyset$ 
34: for each node  $u \in \mathcal{L}_c$  do
35:    $\mathcal{Y}(u) \leftarrow \mathcal{Y}^\downarrow(u) \oplus \mathcal{Y}^\uparrow(u)$   $\triangleright \mathcal{O}(|\mathcal{Y}^\downarrow(u)| |\mathcal{Y}^\uparrow(u)|)$ 
36: end for
37: for each node  $u \in \mathcal{L}_c$  do
38:    $\mathcal{V}_N \leftarrow \text{FILTERDOMINATED}(\mathcal{V}_N \cup \mathcal{Y}(u))$ 
39: end for
40: return  $\mathcal{V}_N$ 

```

Algorithm 2 Top-Down Pareto Frontier Enumeration

Require: Exact decision diagram $\mathcal{D} = (\mathcal{N}, \mathcal{A})$ with layers $\mathcal{L}_1, \dots, \mathcal{L}_{n+1}$

```

1:  $\mathcal{Y}^\downarrow(u) \leftarrow \emptyset$  for all  $u \in \mathcal{N}$ 
2:  $\mathcal{Y}^\downarrow(\mathbf{r}) \leftarrow \{\mathbf{0}\}$   $\triangleright \mathbf{r}$  is the root node in  $\mathcal{L}_1, \mathbf{0} \in \mathbb{R}^m$ 
3: for  $j = 1$  to  $n$  do
4:   EXPANDTOPDOWN( $j, \mathcal{Y}^\downarrow$ )
5: end for
6: return  $\mathcal{Y}^\downarrow(\mathbf{t})$   $\triangleright \mathbf{t}$  is the terminal node in  $\mathcal{L}_{n+1}$ 

```

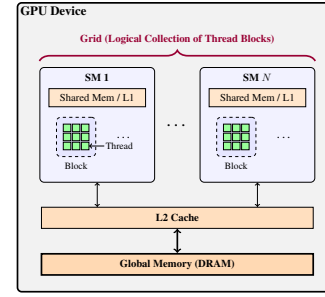


Figure 1: Schematic representation of a GPU architecture.

isters and shared memory, which enables efficient intra-block cooperation and synchronization. SMs connect to a larger, device-wide cache (L2) and ultimately to high-capacity but high-latency global memory (DRAM). Effective GPU implementations therefore aim to expose ample parallelism while limiting expensive global-memory traffic (Nickolls and Dally 2010). This execution model is well suited for workloads dominated by large numbers of independent arithmetic operations and comparisons. In particular, pairwise dominance checks required during PF enumeration in Algorithm 1 are natural candidates for GPU acceleration.

3 Parallel Decision Diagram-based Pareto Frontier Enumeration

This section describes the methodology for leveraging parallelism during the PF enumeration. The different parallelization schemes are introduced in Section 3.1 and their implementation details are provided in Section 3.2.

3.1 Parallelization Schemes

Single thread per node In this scheme, we exploit the inherent independence of nodes within the same layer to achieve parallelism. Specifically, the processing of any node $v \in \mathcal{L}_{j+1}$ relies exclusively on the finalized objective vectors of its predecessor nodes $u \in \mathcal{L}_j$. Because the operations performed on different destination nodes do not mutate or conflict with one another’s memory space, we can process multiple nodes concurrently by assigning a single, dedicated execution thread to each node.

For instance, in the coupled enumeration scheme (Algorithm 1), the execution of EXPANDTOPDOWN (lines 5-10) and EXPANDBOTTOMUP (lines 14-19) can be parallelized over the outer loop of destination nodes. Furthermore, at the cutset layer $\mathcal{L}_c$, the convolution of the top-down and bottom-up frontiers (lines 34-36) can be parallelized across individual nodes since each convolution operates strictly on local label sets $\mathcal{Y}^\downarrow(u)$ and $\mathcal{Y}^\uparrow(u)$ associated with node u . By mapping a single thread per node, this scheme circumvents data races without the need for expensive thread synchronization mechanisms like locks.

Multiple threads per node While assigning a single thread per node is straightforward, the overall enumeration can be further accelerated by utilizing multiple threads to

process a single node collaboratively. However, a naive implementation of such a scheme presents significant synchronization hazards. Consider the EXPANDTOPDOWN procedure, where multiple incoming arcs (u, v) contribute to the same destination node v . A simplistic approach to multi-threading this step would be to assign a separate thread to each incoming arc. Each thread would expand the labels of the parent node of its assigned arc and attempt to merge and filter the generated candidates directly into the destination node’s shared set, $\mathcal{Y}^\downarrow(v)$. This introduces the following critical synchronization issues.

1. **Data Races and Loss:** If threads attempt to update the shared label set concurrently without locks, memory corruption will occur, leading to the unpredictable loss of valid Pareto-optimal candidates.
2. **Synchronization Bottlenecks:** If a lock is applied to the shared label set $\mathcal{Y}^\downarrow(v)$ to prevent data races, the execution essentially serializes, negating the benefits of multi-threading.

To effectively leverage multiple cores per node without succumbing to locking bottlenecks, we can trade off memory for execution speed. Rather than merging candidates into a single shared set on the fly, we can decouple the expansion phase from the filtering phase. First, threads can safely evaluate the addition for all arcs in parallel, scattering the resulting candidate labels into disjoint, pre-allocated memory segments. Once all candidates for the node are materialized in memory, a pool of threads can cooperatively perform the computationally dominant $\mathcal{O}(|\mathcal{Y}|^2)$ dominance filtering operation over this complete set of candidates, allowing multiple threads to actively prune suboptimal vectors simultaneously without locks.

Furthermore, assigning a *fixed* number of threads to collaboratively process each node is suboptimal if the size of the intermediate Pareto frontiers $\mathcal{Y}^\downarrow(v)$ varies drastically across nodes within a given layer. If a static number of threads are allocated per node, the nodes with massive sets of incoming candidates will still bottleneck the overall layer progression, while the thread groups assigned to nodes with few candidates will prematurely finish and sit idle. Thus, an optimal multi-threaded strategy must dynamically allocate a variable number of threads to each node proportional to the size of its candidate label set to ensure an evenly balanced workload. In practice, our CPU implementation realizes the *single thread per node* scheme, while the GPU architecture allows us to efficiently realize the *multiple threads per node* scheme.

3.2 Implementation

Central Processing Unit The CPU implementation directly realizes the single thread per node parallelization scheme. At each layer j , the loop over destination nodes $v \in \mathcal{L}_{j+1}$ is parallelized with a *dynamic work-stealing* schedule: since the number of incoming candidate labels per node varies widely across a layer, a static round-robin assignment would leave many cores idle while others process large frontier sets. Each assigned core processes all incoming arcs (u, v) for its node sequentially, accumulating the

Minkowski sums $\mathcal{Y}^\downarrow(u) \oplus \{\mathbf{v}(a)\}$ and applying the $\mathcal{O}(|\mathcal{Y}|^2)$ dominance filter entirely in thread-local memory, requiring no locks or synchronization. However, a core assigned to a node with a very large local frontier cannot offload work to idle cores, throttling the entire layer’s throughput. This load-imbalance limitation motivates the GPU design below.

Graphics Processing Unit The GPU is well-suited to realize the multiple threads per node scheme: its massively parallel architecture executes thousands of lightweight threads concurrently, making it ideal for absorbing the data-parallelism in candidate generation and dominance filtering. Algorithm 3 presents the GPU-accelerated coupled enumeration, which interleaves top-down and bottom-up layer expansions on the GPU, each decomposed into four data-parallel passes that call the subroutines in Algorithms 4–7.

The FLATTENDD subroutine in Algorithm 3 serializes the entire DD’s arc connectivity for both traversal directions. For each layer $\mathcal{L}_j$, three groups of flat arrays are built. For *top-down* expansion: the source-node index array $b^\downarrow$, the weight-vector array $\mathbf{v}^\downarrow$ (one m -vector per arc), and the per-node incoming-arc offset array $\phi^{a,\downarrow}$ (a CSR-style length- $(|\mathcal{L}_j|+1)$ array whose k -th entry is the index of the first incoming arc of node k). For *bottom-up* expansion: the symmetric arrays $b^\uparrow$, $\mathbf{v}^\uparrow$, and $\phi^{a,\uparrow}$ built from outgoing arcs. The per-node out-degree δ^{out} and in-degree δ^{in} are packed for the scoring heuristic. All arrays are transferred to GPU global memory, amortizing the packing cost across all layer expansions. Back in the main procedure of Algorithm 3, the top-down frontier is initialized with $\mathcal{Y}^\downarrow(\mathbf{r}) = \{\mathbf{0}\}$ at the root and the bottom-up frontier with $\mathcal{Y}^\uparrow(\mathbf{t}) = \{\mathbf{0}\}$ at the terminal, both in GPU memory.

The dynamic layer selection loop executes on the CPU (in Algorithm 3) because each expansion depends on the prior layer’s filtered results. At each step, the algorithm compares the heuristic expansion scores s_{td} and s_{bu} , selecting the direction with the lower score. The score of a layer is computed on the GPU as the total number of candidates that expanding it would generate—the sum over all nodes of the product of the current frontier size and the number of outgoing (top-down) or incoming (bottom-up) arcs. The bottom-up score is scaled by a $1.5\times$ factor to bias the scheme toward top-down expansion, reflecting the empirically higher cost of bottom-up label growth. Within each selected layer expansion, the subroutines EXPANDLAYERTOPDOWN and EXPANDLAYERBOTTOMUP in Algorithm 3 are executed. Both subroutines invoke the same sequence of four GPU passes, using their respective direction-specific packed arrays, as detailed below.

(1) Arc-level offset computation (Algorithm 4): for each arc $a = (u, v)$, one GPU thread reads the source node’s frontier size $|\mathcal{Y}(u)|$, yielding a per-arc candidate count κ_a . An exclusive prefix sum over these counts produces per-arc buffer offsets ϕ_a^e , which determine where each arc’s candidates will be written in the flat buffer β . The per-destination-node candidate total Γ_v is derived from the difference between the edge offsets of the last and first incoming arcs of v .

(2) Candidate expansion (Algorithm 5): the flat buffer β

Algorithm 3 GPU-Accelerated Coupled Pareto Frontier Enumeration

Require: DD $\mathcal{D}$ with layers $\mathcal{L}_1, \dots, \mathcal{L}_{n+1}$

```

1: procedure FLATTENDD( $\mathcal{D}$ )
2:   for  $j = 1$  to  $n + 1$  do
3:     // Top-down: incoming arcs of  $\mathcal{L}_j$ 
4:     Initialize source-node index per arc  $b^\downarrow$ 
5:     Initialize arc weight vector per arc  $\mathbf{v}^\downarrow$ 
6:     Initialize per-node incoming-arc offset  $\phi^{a,\downarrow}$ 
7:
8:     // Bottom-up: outgoing arcs of  $\mathcal{L}_j$ 
9:     Initialize destination-node index per arc  $b^\uparrow$ 
10:    Initialize arc weight vector per arc  $\mathbf{v}^\uparrow$ 
11:    Initialize per-node outgoing-arc offset  $\phi^{a,\uparrow}$ 
12:
13:    // Scoring heuristic
14:    Initialize out-degree per node  $\delta^{\text{out}}$ 
15:    Initialize in-degree per node  $\delta^{\text{in}}$ 
16:  end for
17:  Transfer all packed arrays to GPU global memory
18: end procedure
19: procedure EXPANDLAYERTOPDOWN( $\mathcal{Y}^\downarrow, j$ )
20:   $\kappa, \phi^c, \Gamma \leftarrow \text{COMPUTEARCOFFSETS}(\mathcal{Y}^\downarrow, b^\downarrow, \phi^{a,\downarrow})$ 
21:   $\beta \leftarrow \text{EXPANDCANDIDATES}(\mathcal{Y}^\downarrow, b^\downarrow, \mathbf{v}^\downarrow, \phi^c, \kappa)$ 
22:   $\sigma \leftarrow \text{FILTERDOMINATED}(\beta, \Gamma)$ 
23:   $\mathcal{Y}^\downarrow \leftarrow \text{COMPACTSURVIVORS}(\beta, \sigma)$ 
24:  return  $\mathcal{Y}^\downarrow$ 
25: end procedure
26: procedure EXPANDLAYERBOTTOMUP( $\mathcal{Y}^\uparrow, j$ )
27:   $\kappa, \phi^c, \Gamma \leftarrow \text{COMPUTEARCOFFSETS}(\mathcal{Y}^\uparrow, b^\uparrow, \phi^{a,\uparrow})$ 
28:   $\beta \leftarrow \text{EXPANDCANDIDATES}(\mathcal{Y}^\uparrow, b^\uparrow, \mathbf{v}^\uparrow, \phi^c, \kappa)$ 
29:   $\sigma \leftarrow \text{FILTERDOMINATED}(\beta, \Gamma)$ 
30:   $\mathcal{Y}^\uparrow \leftarrow \text{COMPACTSURVIVORS}(\beta, \sigma)$ 
31:  return  $\mathcal{Y}^\uparrow$ 
32: end procedure
33: FLATTENDD( $\mathcal{D}$ )
34: Initialize on GPU:  $\mathcal{Y}^\downarrow(\mathbf{r}) \leftarrow \{0\}, \mathcal{Y}^\uparrow(\mathbf{t}) \leftarrow \{0\}$ 
35:  $i_{\text{td}} \leftarrow 1; i_{\text{bu}} \leftarrow n + 1$ 
36:  $s_{\text{td}} \leftarrow 0; s_{\text{bu}} \leftarrow 0$ 
37: while  $i_{\text{td}} \neq i_{\text{bu}}$  do
38:   if  $s_{\text{td}} \leq s_{\text{bu}}$  then ▷ Expand top-down
39:      $i_{\text{td}} \leftarrow i_{\text{td}} + 1$ 
40:      $\mathcal{Y}^\downarrow \leftarrow \text{EXPANDLAYERTOPDOWN}(\mathcal{Y}^\downarrow, i_{\text{td}})$ 
41:      $s_{\text{td}} \leftarrow \text{COMPUTESCORE}(\mathcal{Y}^\downarrow, i_{\text{td}})$ 
42:   else ▷ Expand bottom-up
43:      $i_{\text{bu}} \leftarrow i_{\text{bu}} - 1$ 
44:      $\mathcal{Y}^\uparrow \leftarrow \text{EXPANDLAYERBOTTOMUP}(\mathcal{Y}^\uparrow, i_{\text{bu}})$ 
45:      $s_{\text{bu}} \leftarrow 1.5 \times \text{COMPUTESCORE}(\mathcal{Y}^\uparrow, i_{\text{bu}})$ 
46:   end if
47: end while
48:  $c \leftarrow i_{\text{td}}$  ▷ Cutset layer
49:  $\mathcal{Y}_N \leftarrow \text{CUTSETCOUPLING}(\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow, \mathcal{L}_c)$ 
50: return  $\mathcal{Y}_N$ 

```

Algorithm 4 GPU Subroutine: Compute Arc Offsets

Require: Frontier $\mathcal{Y}$; source-node array $b[]$; incoming-arc offsets $\phi^a[]$

```

1: procedure COMPUTEARCOFFSETS( $\mathcal{Y}, b, \phi^a$ )
2:    $M \leftarrow |\mathcal{A}_{\mathcal{L}_j}|$  ▷ Number of arcs in layer  $j$ 
3:   for all arcs  $a = 0$  to  $M - 1$  in parallel on GPU do
4:     ▷ One thread per arc
5:      $\kappa_a \leftarrow |\mathcal{Y}(b(a))|$ 
6:   end for
7:    $\phi_0^c \leftarrow 0$ 
8:   for  $a = 1$  to  $M$  in parallel on GPU do
9:      $\phi_a^c \leftarrow \phi_{a-1}^c + \kappa_{a-1}$ 
10:  end for
11:  for all nodes  $v \in \mathcal{L}_{j+1}$  in parallel on GPU do
12:     $a_{\text{first}} \leftarrow \phi_v^a$ 
13:     $a_{\text{last}} \leftarrow \phi_{v+1}^a$ 
14:     $\Gamma_v \leftarrow \phi_{a_{\text{last}}}^c - \phi_{a_{\text{first}}}^c$ 
15:  end for
16:  return  $\kappa, \phi^c, \Gamma$ 
17: end procedure

```

Algorithm 5 GPU Subroutine: Expand Candidates into Flat Buffer

Require: Frontier $\mathcal{Y}$; source array $b[]$; arc weights $\mathbf{v}(\cdot)$; offsets $\phi^c[]$; counts $\kappa[]$

```

1: procedure EXPANDCANDIDATES( $\mathcal{Y}, b, \mathbf{v}, \phi^c, \kappa$ )
2:   Allocate flat buffer  $\beta$  of size  $\sum_a \kappa_a$ 
3:   for all arcs  $a$  in parallel on GPU do
4:     // One warp (32 threads) per arc
5:     for all  $k = 0, 1, \dots, \kappa_a - 1$  distributed across
6:       warp threads do
7:          $\beta[\phi_a^c + k] \leftarrow \mathcal{Y}(b(a))[k] + \mathbf{v}(a)$ 
8:       end for
9:     end for
10:  return  $\beta$ 
11: end procedure

```

is populated with shifted labels. Each arc a is assigned to a *warp*—a fixed group of 32 GPU threads that execute the same instruction simultaneously, each on different data. The threads within a warp cooperatively iterate over the source node’s labels, with each thread handling one label at a time. Since the prefix sum in Step 1 guarantees that each arc’s output segment $[\phi_a^c, \phi_a^c + \kappa_a)$ is non-overlapping, threads for different arcs write to disjoint memory regions without synchronization.

(3) Load-balanced dominance filtering (Algorithm 6): each destination node v receives $K_v = \lceil \Gamma_v / B \rceil$ thread blocks (where B is the block size). A prefix sum over K_v yields a block-offset array ϕ^k that maps each block to its target node. A tightly packed grid of $\sum_v K_v$ blocks is launched; each block locates its target node via binary search, ensuring no blocks sit idle.

Within each thread block, two levels of the memory hierarchy are exploited. Each thread loads its own candidate y_i from global GPU memory (high-latency DRAM) once

Algorithm 6 GPU Dominance Filtering: Dynamic 1D Load-Balanced Scheduling

Require: Flat candidate buffer β ; per-node candidate counts Γ ; block size B

```

1: procedure FILTERDOMINATED( $\beta, \Gamma$ )
2:    $K[v] \leftarrow \lceil \Gamma[v]/B \rceil$  for all  $v$ 
3:    $\phi_0^k \leftarrow 0$ 
4:   for  $v = 1$  to  $|\mathcal{L}_{j+1}|$  in parallel on GPU do
5:      $\phi_v^k \leftarrow \phi_{v-1}^k + K[v - 1]$ 
6:   end for
7:    $\sigma_i \leftarrow 1$  for all candidates  $i$ 
8:   for all thread blocks  $q = 0, \dots, \sum_v K[v] - 1$  in parallel on GPU do
9:     Binary search  $\phi^k$  to find node  $v$  s.t.  $\phi_v^k \leq q < \phi_{v+1}^k$ 
10:     $\tau \leftarrow q - \phi_v^k$   $\triangleright$  Tile index within node  $v$ 
11:     $[b, e) \leftarrow$  candidate range of node  $v$  in  $\beta$ 
12:     $i \leftarrow b + \tau \cdot B +$  thread index within block
13:    Load  $\mathbf{y}_i \leftarrow \beta[i]$  into per-thread register (if  $i < e$ )  $\triangleright$  Reused across tiles
14:     $\text{dominated} \leftarrow \text{false}$ 
15:    for tile start  $s = b$  to  $e - 1$ , step  $B$  do
16:      Barrier: wait for all threads in block
17:      Each thread  $k$  loads  $\beta[s+k]$  into shared memory slot  $k$  (if  $s+k < e$ )
18:      Barrier: ensure tile is fully visible
19:      if  $i < e$  and not  $\text{dominated}$  then
20:        for  $j = 0$  to  $\min(B, e-s)-1$ , skip  $j = i-b-\tau B$  do
21:          if  $\text{shared}[j] \prec \mathbf{y}_i$  then
22:             $\text{dominated} \leftarrow \text{true}; \text{break}$ 
23:          end if
24:        end for  $\triangleright$  Early exit within tile
25:      end if
26:      if  $\text{dominated}$  then break
27:      end if  $\triangleright$  Early exit across tiles
28:    end for
29:    if  $i < e$  and  $\text{dominated}$  then  $\sigma_i \leftarrow 0$ 
30:    end if
31:  end for
32: return  $\sigma$ 
end procedure

```

Algorithm 7 GPU Subroutine: Compact Survivors

Require: Candidate buffer β of size S ; binary survivor flags σ

```

1: procedure COMPACTSURVIVORS( $\beta, \sigma$ )
2:    $\rho_0 \leftarrow 0$ 
3:   for  $i = 1$  to  $S$  do  $\triangleright$  Parallel prefix sum on GPU
4:      $\rho_i \leftarrow \rho_{i-1} + \sigma_{i-1}$ 
5:   end for
6:   for all candidates  $i = 0$  to  $S - 1$  in parallel on GPU do  $\triangleright$  One thread per candidate
7:     if  $\sigma_i = 1$  then
8:        $\mathcal{Y}(v)[\rho_i] \leftarrow \beta[i]$   $\triangleright v$ : destination node of  $i$ 
9:     end if
10:  end for
11:  return  $\mathcal{Y}$ 
12: end procedure

```

into a *per-thread private register* (line 10), retaining it for all tile comparisons without re-reading global memory. The B block members then cooperatively load a tile of B reference candidates into *shared memory* (fast on-chip scratch-pad visible to all threads in the block), one reference per thread (lines 13–14). Two barrier synchronizations bracket this load so all threads see a consistent tile. Each thread checks $\mathbf{y}_i$ against all B references; a dominance detection immediately breaks both the within-tile and across-tile loops (lines 17–19), avoiding redundant comparisons.

(4) Survivor compaction (Algorithm 7): the dominance filter produces a binary flag $\sigma_i \in \{0, 1\}$ for each candidate i , where $\sigma_i = 1$ indicates that candidate i was not Pareto dominated. An exclusive prefix sum over these flags computes each survivor’s compacted output position ρ_i . A final scatter pass writes only the survivors into the per-node label sets $\mathcal{Y}(v)$.

The top-down GPU enumeration is a degenerate special case of Algorithm 3 in which s_{bu} is never chosen—equivalently, the bottom-up pointer i_{bu} remains fixed at layer $n + 1$. As a result, only EXPANDBLAYERTOPDOWN is invoked at each iteration, and the cutset layer is always $c = n + 1$. At that point, $\mathcal{Y}^\downarrow(\mathbf{t})$ already constitutes the complete Pareto frontier, so CUTSETCOUPLING trivially returns $\mathcal{Y}^\downarrow(\mathbf{t})$.

Once the two frontiers meet at the cutset layer $\mathcal{L}_c$, the final convolving and merging step is delegated to the GPU subroutine CUTSETCOUPLING (Algorithm 8). This subroutine filters for active nodes $\mathcal{V}_c^* \subseteq \mathcal{L}_c$ that possess non-empty frontiers in both directions (line 1), since nodes lacking either cannot produce feasible complete paths. To maximize GPU execution efficiency and prevent warp divergence, the active nodes are sorted in descending order of their total frontier sizes $\text{score}(v) = \kappa_v^\downarrow + \kappa_v^\uparrow$. They are then processed in load-balanced, memory-bounded batches $\mathcal{B}$ s.t. the total number of Cartesian product candidates generated per batch does not exceed B_{max} (default 625,000).

For each batch $\mathcal{B}$, a custom GPU kernel materializes the Minkowski sum candidates β (Algorithm 9). Since convolving clean frontiers can generate redundant duplicates or

Algorithm 8 GPU Subroutine: Cutset Coupling

Require: Cutset nodes $\mathcal{L}_c$; frontiers $\mathcal{Y}^\downarrow(v), \mathcal{Y}^\uparrow(v)$ and sizes $\kappa_v^\downarrow, \kappa_v^\uparrow$ for all $v \in \mathcal{L}_c$; candidate batch limit $B_{\max}$

- 1: $\mathcal{V}_c^* \leftarrow \{v \in \mathcal{L}_c \mid \kappa_v^\downarrow > 0 \text{ and } \kappa_v^\uparrow > 0\}$ $\triangleright$ Active nodes
- 2: $\text{score}(v) \leftarrow \kappa_v^\downarrow + \kappa_v^\uparrow$ for all $v \in \mathcal{V}_c^*$
- 3: Sort $\mathcal{V}_c^*$ in descending order of $\text{score}(v)$
- 4: $\mathcal{Y}^{\text{global}} \leftarrow \emptyset$ $\triangleright$ Running global frontier in GPU memory
- 5: Partition sorted $\mathcal{V}_c^*$ into batches $\mathcal{B}_1, \mathcal{B}_2, \dots$ s.t. $\sum_{v \in \mathcal{B}_k} \kappa_v^\downarrow \cdot \kappa_v^\uparrow \leq B_{\max}$
- 6: **for** each batch $\mathcal{B}$ **do**
- 7: $\beta \leftarrow \text{MATERIALIZEPRODUCTS}(\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow, \mathcal{B})$
- 8: **if** $\mathcal{Y}^{\text{global}} \neq \emptyset$ **then**
- 9: $\sigma \leftarrow \text{FILTERAGAINSTFRONTIER}(\beta, \mathcal{Y}^{\text{global}})$
- 10: $\beta \leftarrow \text{COMPACTSURVIVORS}(\beta, \sigma)$
- 11: **end if**
- 12: $\beta \leftarrow \text{SELFPRUNE}(\beta)$ $\triangleright$ Local 1D dominance check
- 13: $\mathcal{Y}^{\text{global}} \leftarrow \text{MERGEFRONTIERS}(\mathcal{Y}^{\text{global}}, \beta)$ $\triangleright$ Mutual pruning and concatenation
- 14: **end for**
- 15: **return** $\mathcal{Y}^{\text{global}}$

Algorithm 9 GPU Subroutine: Materialize Cutset Products

Require: Frontiers $\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow$; node offsets $\phi^\downarrow, \phi^\uparrow$ and sizes $\kappa^\downarrow, \kappa^\uparrow$ for nodes in batch $\mathcal{B}$; product offsets ϕ^p

- 1: **procedure** $\text{MATERIALIZEPRODUCTS}(\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow, \mathcal{B})$
- 2: Allocate flat candidate buffer β of size $\sum_{v \in \mathcal{B}} \kappa_v^\downarrow \cdot \kappa_v^\uparrow$
- 3: **for all** nodes $v \in \mathcal{B}$ **in parallel on GPU do** $\triangleright$ One thread block per node
- 4: $T \leftarrow \kappa_v^\downarrow \cdot \kappa_v^\uparrow$ $\triangleright$ Total candidates for node v
- 5: **for all** candidate indices $p = 0, \dots, T - 1$ **in parallel do**
- 6: $i \leftarrow \lfloor p / \kappa_v^\uparrow \rfloor$; $j \leftarrow p \pmod{\kappa_v^\uparrow}$
- 7: $y^\downarrow \leftarrow \mathcal{Y}^\downarrow(\phi_v^\downarrow + i)$; $y^\uparrow \leftarrow \mathcal{Y}^\uparrow(\phi_v^\uparrow + j)$
- 8: $\beta[\phi_v^p + p] \leftarrow y^\downarrow + y^\uparrow$
- 9: **end for**
- 10: **end for**
- 11: **return** β
- 12: **end procedure**

dominated points, each batch undergoes a three-stage filtering pipeline on the GPU: (1) filtering against the running global frontier $\mathcal{Y}^{\text{global}}$ (line 9) to prune candidates dominated by already found solutions, (2) local self-pruning (line 11) using the 1D dominance filter to remove intra-batch dominated points, and (3) mutual merging (line 13) which filters the running global frontier against the batch survivors, and then concatenates the remaining candidates.

References

Ben-Nun, T.; and Hoefer, T. 2019. Demystifying parallel and distributed deep learning: An in-depth concurrency analysis. *ACM Computing Surveys (CSUR)*, 52(4): 1–43.

Bergman, D.; Bodur, M.; Cardonha, C.; and Cire, A. A. 2022. Network models for multiobjective discrete optimization. *INFORMS Journal on Computing*, 34(2): 990–1005.

Ciré, A. A. 2018. Decision diagrams for optimization. *Constraints*, 20: 494–495.

Deb, K.; Agrawal, S.; Pratap, A.; and Meyarivan, T. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.*, 6: 182–197.

Ehrgott, M. 2005. *Multicriteria optimization*. Springer.

Ehrgott, M.; Gandibleux, X.; and Przybylski, A. 2016. Exact methods for multi-objective combinatorial optimisation. In *Multiple criteria decision analysis: State of the art surveys*, 817–850. Springer.

Garroppo, R. G.; Giordano, S.; and Tavanti, L. 2010. A survey on multi-constrained optimal path computation: Exact and approximate algorithms. *Computer Networks*, 54(17): 3081–3107.

Lindholm, E.; Nickolls, J.; Oberman, S.; and Montrym, J. 2008. NVIDIA Tesla: A unified graphics and computing architecture. *IEEE micro*, 28(2): 39–55.

Miettinen, K. 1999. *Nonlinear multiobjective optimization*, volume 12. Springer Science & Business Media.

Nickolls, J.; Buck, I.; Garland, M.; and Skadron, K. 2008. Scalable parallel programming with cuda: Is cuda the parallel programming model that application developers have been waiting for? *Queue*, 6(2): 40–53.

Nickolls, J.; and Dally, W. J. 2010. The GPU computing era. *IEEE micro*, 30(2): 56–69.

Vianna, D.; and Arroyo, J. 2004. A GRASP algorithm for the multi-objective knapsack problem. In *XXIV International Conference of the Chilean Computer Science Society*, 69–75.